

Analysis

Duncan Parke

Data Structures

This lab uses strings and `istringstream`s for processing of the prefix expressions. These were used because the input is already stored as strings, making it easy to read in, and the `istringstream` object just makes token extraction simpler, allowing sequential characters to be stripped from the string input

Also, we use the implicit data structure of the call stack, which functions like a build stack in that it allows us to keep information stored in a hierarchal way. Using it is particularly appropriate for this application, because we're working with data in a hierarchal way, so since we have to process operators and operands differently in a deterministic manner, we have a good foundation for using the data structures of strings, `stringstream`s, and the call stack to process our data.

Design

Generally, our design just checks the base case - an empty string, and returns empty if the string is empty, triggering the full call stack build. If it is not empty, and the character streamed in is of type operator, we return a recursive call on the two operands that follow it in the string stream. Because parsing prefix has a format of operator operand operand, this will allow us to go down the left and right trees to resolve any substrings which follow the operand

The recursive design is $O(n)$ time efficient, where n is the number of characters in the line. This is because each character is only evaluated once.

The design is also $O(n)$ in the space domain, because it copies the character using the call stack, which is the minimum case of recursion anyhow.

Comparing against Iterative

The recursive approach is a bit more natural, especially if you think of the prefix function as a tree. However, the iterative function will scale better because while there is a definite benefit in code readability, and they share the same time complexity, the recursive function is less memory efficient and may have a stack overflow at high string lengths. In programming languages like python, it will just stop working at a call stack depth of 1000, meaning that the recursive function is not the optimal solution.

What I learned

This exercise was a good reminder of how recursion can simplify problems, but isn't always optimal for performance. Since IO worked through both recursive and iterative approaches, which just reinforced that the right approach depends on the specific situation—recursive for simplicity, iterative for efficiency. It's clear that recursion can be elegant but has its limits with deep recursion or large inputs. Overall, just another exercise in balancing readability with performance.

What I might do differently

In the future, I might have tried to make the `convert_to_postfix_recursive` function not require a jumping off point, and just be a single function. I didn't modify it in order to reduce the amount of modification I made to the original file. I also would try to handle the string/char paradigm better, possibly by just using a different language

Enhancements

The program can handle inconsistent spacing, which helps deal with errant user input Also - I build a testing suite (though its from last assignment as well), which builds test suite files and checks them automatically using randomization.